

19 — Key blocks: a key change never updates, it rebuilds

KEY BLOCKS STATE COMPONENTS

Q. I switched articles and my half-typed comment draft vanished — I only changed the key, so why did everything inside rebuild?

ANSWER. Because `{#key}` never updates anything. When the key expression changes, Svelte destroys everything inside the block — the component instance, its local state, your draft — and builds a brand-new instance from zero. A key change is not a refresh; it is a teardown followed by a fresh start.

WHY IT WORKS. Every normal state change in Svelte patches the DOM in place and preserves component state, so it is natural to expect a key change to behave the same way. But the key exists to declare identity: a new key means “this is a different thing,” and Svelte takes you literally, throwing the old instance away so no stale state leaks into the new one.

ArticlePage.svelte

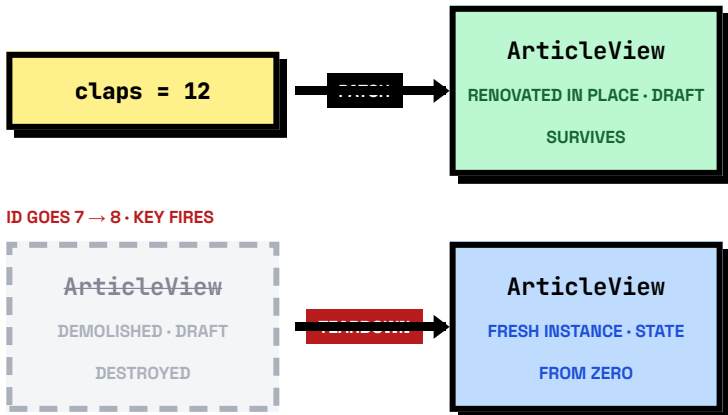
```
<script>
  let { article } = $props();
</script>

{#key article.id}
  <ArticleView {article} />
  <!-- new id → old instance destroyed, fresh state,
transitions replay -->
{/key}
```

DIAGRAM CONCEPT. The Wrecking Ball: a normal state change is a renovation — the building stays, the furniture stays; a key change is a demolition at the same address — old instance razed, new one erected, nothing inside survives.

THE WRECKING BALL

ID STAYS 7 • NORMAL UPDATE



- ① **Summary.** A key change is a demolition, not a refresh: everything inside `{#key}` is destroyed and rebuilt with fresh state. Reach for it when switching to a new article should feel like opening a clean page — and expect anything unsaved inside to be gone.